

meArm 机械臂 固件设计说明

评审向文档 —— 不阅读源代码即可理解程序的整体逻辑

项目	内容
项目名	meArm 四自由度桌面机械臂
控制器	Arduino Uno (ATmega328P, 16 MHz, flash 可用 32256 B, SRAM 2048 B)
本说明对应版本	v1.6.3 (tag 7a1d818, 含单文件整合产物)
代码仓库	https://github.com/2122019145/meArm.git
代码规模	21 个参与编译的源文件, 约 5 900 行; 自检脚本与探针约 4 900 行
文档性质	面向评审: 先讲整体逻辑与数据流, 再讲各板块实现与取舍, 最后讲专题 (示教点记录/轨迹生成/速度控制/异常处理) 与未完成项分析

本文档的取材方式: 所有结论均来自仓库内源码、自检脚本与可复现的编译/探针输出 (见第 8.4 节与附录 B)。文档中每一个数字都可在仓库中复核, 涉及实测的结论均标注了实测条件; 未上机验证的内容在第 7 章明确列为未完成项。

0 摘要与阅读指引

0.1 这个固件做什么

固件在一块 Arduino Uno 上同时提供四件事:

- 1) **手动运动:** 两根摇杆各自两个方向, 共四路, 分别驱动基座回转、上臂俯仰、下臂俯仰、夹爪开合; 四路可同时动作, 不需要切换模式。
- 2) **自动取放:** 收到 A/B/C 三个命令之一后, 机械臂按“移动到物体上方 → 垂直下降 → 合爪 → 停顿 → 垂直抬起 → 平移到放置点上方 → 垂直下降 → 松爪 → 停顿 → 垂直撤离”的顺序, 把一个物体从初始位置搬到放置位置。
- 3) **绘制:** 把铅笔夹在末端, 把纸面高度告诉它之后, 可以在纸上画一条直线、一个字母 V, 或者由操作者用摇杆“示教”五个点后, 画出折线或圆滑曲线; 绘制过程可暂停、继续、取消。
- 4) **录制回放:** 把一次手工操作 (摇杆推出来的动作) 录成轨迹, 之后可一键重放; 也可一键回到开机待机位姿。

四件事共用同一套底层: 几何反解、关节限位、点位运动与舵机输出。所有动作都不会阻塞主循环——任何时刻串口仍然响应、指示灯仍然闪烁、摇杆仍然被采样。

0.2 三条贯穿全局的设计不变量

读代码之前先记住三句话, 后面所有“为什么这样写”都能对得上:

- 1) **被控量是四个关节角, 末端坐标是算出来的派生量。**程序内部唯一的**状态**是 `Pos.ser` (b/r/c/f 四个角度), `Pos.rec` (末端 x/y/z) 由正运动学实时算出, 只用于显示、绘图与边界判断。因此摇杆走的是“角度”, 而绘图/取放走的是“坐标”, 两条路径最终都落到同一组角度上。
- 2) **失败路径绝不改状态。**取放和绘图在动作开始前会把整条路径采样校验一遍, 任一点不合法就拒绝启动; 点位运动在一轮里解算失败就整点丢弃、本轮不推进轨迹; 关节步进在越界时整步回退。结果是: 任何被拒绝的请求, `Pos` 一个字节都没变, 机械臂停在原地。
- 3) **同一时刻只有一个“源”在驱动机械臂。**取放序列、录制回放、绘图任务互斥; 冲突时后来者得到 `BUSY` 而不是“抢方向盘”。唯一的例外是调速命令——任何状态下 H/L/1/2/3 都有效, 方便随时把速度降下来。

0.3 阅读指引

文档结构与对应内容

章	内容	适合谁看
第 1 章	系统总览：硬件、分层、主循环、编译开关与容量预算	想先建立全局印象的评审
第 2 章	各板块实现逻辑：每个模块的职责、数据、算法与设计取舍	要逐项核对实现的人
第 3 章	专题：示教点记录的三种通道及其数据结构	关心“示教怎么存”的人
第 4 章	专题：轨迹生成的四种来源与完整校验链	关心“路怎么走出来”的人
第 5 章	专题：速度控制的三层结构与各自动作的速度参数	关心“为什么这么快/这么慢”的人
第 6 章	专题：异常处理的分类、检测点与对外表现	关心“出错会怎样”的人
第 7 章	未成功能清单与原因分析（含改进路径）	评估完成度的人
第 8 章	评审速查：命令表、回复码、关键常量、源码索引、验证方法	要现场抽查的人

1 系统总览

1.1 硬件与运行环境

硬件资源分配

资源	用途	说明
D9 / D7 / D8 / D6	舵机 b / r / c / f	基座回转、上臂俯仰、下臂俯仰、夹爪开合；逻辑角统一 0~180°
A0 / A1	左手柄 X / Y	基座（角度增大）、上臂俯仰（前推减小）
A2 / A3	右手柄 X / Y	夹爪（角度增大=张开）、下臂俯仰（前推减小）
D13	板载 LED	运动指示：静止灭、运动期间 100 ms 闪烁
D2~D5	四个独立按键（可选）	默认 不启用 （本机只接摇杆），功能改由串口 N/R/P/M 触发，详见 1.4 与 7.2
D0 / D1	串口 RX / TX	115200 8N1，唯一上位机通道
Timer1	4 路舵机脉冲发生器	自研驱动，替代 Arduino Servo 库（见 2.2）
USART RX 中断	串口接收	字节进 64 字节环形缓冲，主循环取行（见 2.9）
flash	程序 + 常量表	21 个源文件；默认配置占 31656 B（96.6%）
SRAM	状态 + 录制缓冲	默认占 1415 B（69.1%），其中录制缓冲固定 768 B

关于容量：Uno 的 32 KB flash 里有 512 B 由 bootloader 占用，**avr-size** 报告的百分比按 32768 B 计算，因此“96.6%”是相对 32768 而言的；真正可用的程序空间是 **32256 B**，默认配置下还剩 **600 B**。这个数字是理解第 1.4 节与第 7 章所有取舍的钥匙。

1.2 软件分层与模块清单

固件按“依赖只向下”分层，共 21 个参与编译的文件（不含自检）。依赖关系是单向的：上层可以调用下层，下层绝不回头调用上层。

模块清单（行数为 v1.6.3 实测）

层	文件	行数	职责
应用	meArm.ino	244	初始化与主循环调度；唯一的舵机写出点 writeServo()
功能	pick_place.cpp/.h	509 / 83	A/B/C 自动取放状态机
功能	draw_control.cpp/.h	1388 / 274	绘图：任务表、示教、轨迹、速度规划、校验
功能	button_control.cpp/.h	782 / 137	按键与串口触发的录制/回放/回中/循环取放
功能	joystick_control.cpp/.h	477 / 133	摇杆采样、中位自标定、四路关节步进
运动	move.cpp/.h	223 / 145	关节步进 moveJointStep() 与坐标运动 moveToPoint()
运动	path_core.h	105	路径几何核心：单点校验、解算提交、线段采样校验（header-only）
几何	constant_and_positions.cpp/.h	896 / 244	姿态状态、限位、正解/反解、全局调速参数
执行器	servo_drive.cpp/.h	129 / 75	Timer1 顺序脉冲四路舵机驱动（替代 Servo 库）
协议	serial_protocol.cpp/.h	973 / 128	串口收发、命令分发、回复层、运行时功能开关
协议	protocol_constants.cpp/.h	66 / 102	命令字符、轴映射、行缓冲与波特率、速度档位边界
配置	weArm_config.h	226	编译期功能开关与容量/录制深度参数

为什么把 path_core.h 单独抽出来：绘图（draw_control.cpp）与串口坐标移动原本各有一套“点是否合法 / 解算并提交 / 线段采样是否安全”的代码，两边一旦不一致就会出现“绘图能过、串口过不去”的诡异差异。v1.6.0 把这段几何核心抽成 header-only 的 inline 函数，成为全工程唯一实现（见 2.4），删除的旧函数包括绘图里的 solveStep()/tryApplyPoint()/applyPointClamped()/solveJoint() 与串口里的 protoMoveToCoords()，默认配置因此净省 168 B flash。

1.3 主循环与调度顺序

初始化（setup）：

- 1) posInit() —— 把姿态复位到工作空间内的安全初始点并解算一次，保证后续模块拿到的 Pos 是自洽的；
- 2) serialProtocolBegin() —— 按 115200 初始化串口并打印命令表（必须早于其它会往串口输出的初始化）；
- 3) joystickSetup() —— 配置 A0~A3 为输入并做中位自标定（此刻机械臂尚未动作，没人碰摇杆，标定最干净）；
- 4) buttonSetup() / drawSetup() —— 初始化按键与绘图模块的软件状态；
- 5) servoDriveBegin() → servoDriveAttach(0,9) / (1,7) / (2,8) / (3,6) → writeServo() —— 启动 Timer1 脉冲发生器、绑定四个通道、把初始姿态写出去。

主循环（loop）， 每轮按固定顺序调用一次，全部为非阻塞函数，函数内部不使用 delay()：

- 1) serialProtocolLoop() —— 收一行、执行一行、回一行；
- 2) pickPlaceLoop() —— 取放序列推进一步（空闲时立即返回）；
- 3) buttonLoop() —— 按键扫描、录制采样、回放/回中推进一步；
- 4) drawLoop() —— 绘图状态机推进一步（空闲时立即返回）；
- 5) joystickLoop() —— 摇杆采样并按档位步进（被别的模块占用时让位）；
- 6) writeServo() —— 把四个逻辑角写进舵机通道（唯一写出口）。

为什么“唯一写出口”很重要：实机上基座（通道 0）与夹爪（通道 3）的舵机是反向安装的，逻辑角增大物理上反而反向。固件没有去改每个模块的角度语义，也没有改 0~180 的映射，而是在 writeServo() 里对这两个通道做一次镜像：physical = (min + max) - logical（在自己行程窗口内镜像）。好处是：全工程只有这一处知道“装反了”，任何模块（摇杆、取放、绘图、回放）都按统一的方向语义工作；将来换成正装舵机，把两个编译开关改成 0 即可。夹爪行程是 [60, 150]（60+150=210），镜像后仍落在 [60, 150] 内，不会把夹爪推进行程外。

1.4 编译期功能开关与容量预算

所有开关集中在 weArm_config.h，默认值即出厂配置。开关的实现方式是：关掉某功能时，该模块的头文件退化为一组内联空桩函数，调用方（主循环、串口、摇杆、按键）一行都不用改。

编译期开关（默认值即当前出厂配置）

开关	默认	作用与取舍
WEARM_ENABLE_PICK_PLACE	1	A/B/C 自动取放；关掉省约 12.5 KB flash
WEARM_ENABLE_BUTTONS	1	录制/回放/回中/循环取放；关掉省约 3.2 KB flash
WEARM_ENABLE_DRAW	1	绘图；关掉省约 9.3 KB flash
WEARM_BUTTON_PINS	0	是否读 D2~D5 四个物理按键。本机不接独立按键，置 0 后连 pinMode 都不做（省 510 B flash / 28 B SRAM）；四键功能改由串口 N/R/P/M/O 触发

WEARM_MIRROR_BASE / _TOOL	1 / 1	基座与夹爪舵机反向安装的逻辑镜像（见 1.3）
WEARM_REC_TICK_MS	200	录制采样周期（ms）：每 200 ms 存一条记录
WEARM_REC_ENTRIES	192	录制缓冲条数，每条 4 字节 $\Rightarrow$ 占 768 B SRAM，单次录制上限 $192 \times 200 \text{ ms} = 38.4 \text{ s}$
WEARM_DEBUG_SERIAL	0	打开后所有模块的逐步骤调试追踪走 Arduino Serial，flash 增加约 9260 B（当前配置根本装不进 Uno）
WEARM_SERIAL_ARDUINO	未定义	定义后串口退回官方 HardwareSerial 后端（省下的 852 B 吐回去），仅在排查串口问题时使用

四种功能组合的实际体积（avr-size 实编，百分比按 32768/2048）

配置	Program	Data	可用于其它功能
取放 + 按键 + 绘图（默认）	31656 B (96.6%)	1415 B (69.1%)	flash 余 600 B
取放 + 绘图（按键关）	28478 B (86.9%)	498 B (24.3%)	flash 余 3778 B, SRAM 余 1550 B
取放 + 按键（绘图关）	19198 B (58.6%)	1206 B (58.9%)	flash 余 13058 B
绘图 + 按键（取放关）	28818 B (87.9%)	1381 B (67.4%)	flash 余 3438 B

真正让“三功能全开 + 四按键引脚全部可用”成为可能的是两件事：① 弃用 Arduino 的 `HardwareSerial`——只要还链接 `Serial` 对象，它的虚表就会把整个 `HardwareSerial` 拖进镜像，逐函数裁剪做不到；改成直接操作 `UDR0/UCSR0A` 并让接收走 `USART RX` 中断后，整块省下 852 B flash / 109 B SRAM。② 自研 4 路舵机驱动取代 `Servo` 库，省约 550~650 B（见 2.2）。反过来说：默认配置只剩 600 B 余量，任何新功能都必须先省出空间，这是第 7 章的贯穿性约束。

1.5 数据流：一条命令从进入到动起来

以串口输入 `x20,y0,z40` 为例，完整链路如下（这也是理解全部模块协作的最短路径）：

- 1) `USART RX` 中断把每个字节放进 64 字节环形缓冲；主循环 `serialProtocolLoop()` 取字节攒成一行（行缓冲 40 字节，见到 `\n` 或静默 300 ms 即认为整行到齐）；
- 2) `protoHandleLine()` 识别出“坐标指令”，把用户坐标系（原点在地面垂足）的 `z` 减去肩高 `WEARM_SHOULDER_HEIGHT(20.0)`，得到内部坐标系（原点在肩关节）的 `(x, y, z)`；
- 3) 其余两轴沿用当前 `Pos.rec`（`seen` 位掩码记录哪几轴被提及），组成目标点；
- 4) `moveToPoint(goal, seen, 0, 0, MOVE_XYZ_NOW)` —— 即时到位策略：反解出关节角、一次写入 `Pos.ser` 并刷新 `Pos.rec`；反解失败（不可达、地面以下）则返回 `REJECTED`，`Pos` 一个字节不改；
- 5) `writeServo()` 把四个逻辑角写进舵机通道（基座/夹爪做镜像）；
- 6) `serialProtocolLoop()` 回一行 `OK`（失败时回 `REJECTED`，机械臂正忙时回 `BUSY`）。

再以摇杆推一下为例：`joystickLoop()` 采样 `A0~A3`、扣掉死区、判断哪一路偏转最大 $\rightarrow$ 按偏转量算出这一格的步进间隔 $\rightarrow$ 调用 `moveJointStep()` 让对应关节转一个 `speed.stepSize` $\rightarrow$ 越界则整步回退（表现为“推到头就停住”） $\rightarrow$ `writeServo()` 输出。整个过程里 `Pos.rec` 是被重算出来的，不是被直接写的。

2 各板块实现逻辑

本章逐模块说明：职责 → 数据结构与对外接口 → 实现逻辑 → 关键取舍。每小节末尾给出对应代码位置，便于抽查。

2.1 姿态与限位核心 `constant_and_positions`

2.1.1 职责与数据结构

本模块是整个固件的“地面真值”：它持有机械臂的当前姿态、两级边界、正/逆运动学，以及全局调速参数。其余模块只通过它读写姿态，不自己维护角度副本。

核心数据结构

名字	内容	作用
<code>struct SER</code>	<code>angle1 b / angle2 r / angle3 c / angle4 f</code>	四个关节角（度），是被控量
<code>struct REC</code>	<code>x / y / z</code>	末端坐标（内部坐标系，原点在肩关节），是派生量
<code>struct pos</code> （全局 <code>Pos</code> ）	<code>SER ser; REC rec;</code>	当前姿态； <code>ser</code> 为准， <code>rec</code> 由正解保持同步
<code>struct arm</code> （ <code>arm1</code> ）	<code>L1=20、L2=20、armheight=0</code>	连杆长度与肩高（单位与坐标一致）
<code>struct rangeLimit</code> （ <code>limit</code> ）	<code>minX/maxX/minY/maxY/minZ/maxZ</code>	末端坐标的粗边界（工作空间）
<code>struct servoLimitCfg</code> （ <code>servoLimit</code> ）	<code>b/r/c/f</code> 各自 <code>min/max</code>	关节角行程，最终防线
<code>struct speedCfg</code> （ <code>speed</code> ）	<code>stepSize / minDelayMs / fullDelayMs</code>	全局调速：每格角度、最快间隔、最慢间隔

2.1.2 两级边界：为什么需要两套

只有一套关节限位是不够的：关节在行程内，末端完全可能跑到桌子底下或伸出工作空间。只有一套坐标边界也不够：坐标合法，反解出来的关节角可能已经顶到机械限位。所以固件用两套、分工明确：

- **`servoLimit`（关节行程，细边界、最终防线）**：`b`、`r`、`c` 各 $0^{\circ}\sim 180^{\circ}$ ，夹爪 `f` 为 $60^{\circ}\sim 150^{\circ}$ 。任何写入 `Pos.ser` 的路径都会先把角度夹到行程内（CLAMP 模式）或直接拒绝（REJECT 模式）。
- **`limit`（末端坐标，粗边界、挡得快）**：当前配置 $x \in [-40, 40]$ 、 $y \in [-40, 40]$ 、 $z \in [-20, 40]$ ，与“关节行程算出来的真实可达包络”完全一致、余量 0（由自检脚本穷举验证）。它的作用是让明显越界的请求在解算之前就被挡掉。

两个细节值得一提。其一，`limit` 的配置本身会被检查：`rangeClampConfig()` 对每一轴做排序，把“min 写成比 max 大”的配置就地改正并统计条数——避免因为一个写反的数字把整台机器锁死。其二，越界并不是“报错”，而是“这个方向到头了”：摇杆推到头时表现为停住，而不是弹错误。

2.1.3 正运动学与逆运动学

正解（`recFromServo()`）：把末端从角度算成坐标。平面内两条连杆首尾相接， $x_{\text{planar}} = L1 \cdot \cos \alpha + L2 \cdot \cos \beta$ 、 $z = L1 \cdot \sin \alpha + L2 \cdot \sin \beta + \text{armheight}$ ，其中 $\alpha = r$ 、 $\beta = r - c$ ；再绕竖直轴旋转 $\theta = 90^{\circ} - b$ 得到 $x = x_{\text{planar}} \cdot \cos \theta$ 、 $y = x_{\text{planar}} \cdot \sin \theta$ 。注意 `angle4`（夹爪）不参与——它不影响“铅笔尖/夹爪中心”这个被控点，只影响开合。

逆解（`getAngleEx()`）：给定坐标求四个角。基座角是唯一确定的： $b = 90^{\circ} - \text{atan2}(y, x)$ ；平面内两连杆问题有两个解（肘上/肘下），再加上“基座是否反折 180° ”的可能，固件枚举 4 个候选分支： $c = \pm \text{acos}((R^2 - L1^2 - L2^2)/(2 \cdot L1 \cdot L2))$ ， $\alpha = \text{atan2}(z - \text{armheight}, p) +$

$\text{atan2}(L2 \cdot \sin c, L1 + L2 \cdot \cos c)$ ，其中反折分支把平面坐标取负。每个候选用正解回代算残差，取残差最小者；残差大于 0.05 就整组丢弃，返回失败。

为什么必须枚举反折分支：臂长 20+20，如果只考虑“朝前的解”，基座旋转超过 90° 之后的一整片区域会突然无解或跳到别扭姿态。枚举反折分支后，机械臂可以在左右两侧都保持自然姿态，这也是取放位置表能同时用到 x 正负两侧的原因。

2.1.4 角度规范化与容差

- `normRevAngle()`：把角度规整到 $[0^\circ, 360^\circ)$ ，把“360 减去一个极小量”归零。没有这一步，反解在奇异点附近会产出 359.9999° 这类角，被限位判为越界而误拒一个本来合法的姿态。
- `ANGLE_EPS = 1e-6`：角度比较的容差，专治 $-1e-15$ 与 -0.0 这类浮点残渣。
- `RANGE_EPS = 1e-9`：坐标“贴边界”判定容差，让“恰好等于边界”的点被视为可达（否则 40.0000000001 与 40 会得到不同结论）。

2.1.5 标定与自检

正解的正确性用两个已知位形自检： $(r, c) = (90, 90)$ 时末端应为 $(20.00, 20.00)$ ； $(r, c) = (90, 105)$ 时末端应为 $(19.32, 14.82)$ （下臂角增大，末端往前同时往下）。自检脚本还做了 20 万组“随机关节角 → 正解 → 反解 → 正解”的往返验证：0 组失败，最大位置误差 $2.266e-10$ （双精度下的数值噪声）。

另有 `servoSelfCheck()`：检查四个关节的 $\min \leq \max$ 且都落在 $0 \sim 180$ 内，把不合法项就地修正并返回修正条数；`posInit()` 在启动时把姿态复位到一个工作空间内的安全点。

代码位置：`constant_and_positions.h:1-244`（常量/结构/接口）、`constant_and_positions.cpp:37`（`arm1` 初值）、`:349-371`（`isReachable`）、`:395-401`（`fkPlanar`）、`:403-428`（`recFromServo`）、`:514-524`（`ikBranch` 分支枚举与残差）、`:533-538`（`normRevAngle`）、`:540+`（`getAngleEx`）。

2.2 舵机驱动 servo_drive

2.2.1 为什么要自己写

Arduino 的 Servo 库是全平台通用实现，为“任意通道数、任意引脚、运行期 attach/detach、读回角度、多定时器实例”付出代价；本工程只用其中极小一部分（固定 4 路、固定引脚、只写 $0 \sim 180^\circ$ 、不 detach、不读回）。在 ATmega328P 上实测占用：`Timer1` 中断向量 376 B + `Servo::write` 248 B + `Servo::attach` 240 B + `turnOffPWM` 82 B + `Servo` 全局数组静态构造 40 B $\approx$ 986 B；自研实现约 300 B，净省约 650 B。

2.2.2 行为必须逐位一致

换驱动的前提是“不动机械行为”。旧代码走 `servos[i].write((int)angle)`，实际链路是：角度夹到 $0 \sim 180 \rightarrow \text{map}(\text{angle}, 0, 180, 544, 2400) = \text{angle} \times 1856 / 180 + 544$ （32 位长整型运算） $\rightarrow$ 脉宽再夹到 $[544, 2400] \rightarrow$ 减去 `TRIM_DURATION` (2 μs) $\rightarrow \text{usToTicks}()$: $(16 \text{ MHz} \times \mu\text{s}) / 8 = \mu\text{s} \times 2$ 。所以最终写入 `Timer1` 的比较值 = ((脉宽 μs - 2) $\times$ 2) 个 tick，1 tick = 0.5 μs （预分频 8）。自研驱动的 `servoDriveTicksForDeg()` 逐位复刻这条链；因为 `servoLimit` 保证角度恒在 $0 \sim 180$ ，`Servo` 库中“ ≥ 544 当作微秒”的分支不会被走到，故不实现。

2.2.3 时序：20 ms 帧与一个真实的坑

每个 20 ms 帧内，按通道 0,1,2,3 顺序连续输出 4 个脉冲（脉冲之间不留空隙，与 `Servo` 库的 `handle_interrupts()` 一致），4 路发完后等待帧尾。`Timer1` 跑普通模式（比较匹配不清零计数

器），所有比较值都是**绝对计数**：帧边界把 TCNT1 清零并在同一次中断里就拉起第 0 路，也就是说“帧边界 = 第 0 路脉冲上升沿”，帧长因此严格是 40000 tick = 20 ms。

这个设计绕过了一个真实的下溢 bug：帧尾的比较值必须在“4 路都发完”的那一次中断里就算好（等于绝对计数 40000）；如果等到帧尾中断里现算“还差多少”，中断入口读到的计数器已经越过 40000，uint16 减法下溢会把下一帧推到约 52.8 ms 之后——帧长变成 19 Hz 并且抖动，四个舵机会同时发出异响。

2.2.4 可测性

硬件相关代码只有 servoDriveBegin() 里的 5 行寄存器操作和一个中断壳子（用 #if defined(__AVR__) 隔开）；脉冲状态机 servoDriveStep() 是纯函数：给它“当前计数器值”，它决定本次事件拉高/拉低哪个引脚并返回下一次比较值。PC 端探针 probe_servo_drive 直接按 tick 步进这个状态机，用 mock 的 digitalWrite 记录，验证“引脚顺序 + 脉宽 + 帧长”。因此时序逻辑不是“只能上硬件才能验证”。

代码位置：servo_drive.h:1-75（含完整设计说明）、servo_drive.cpp（129 行，硬件寄存器 + ISR 壳子 + 纯状态机）。

2.3 点位运动核心 move

2.3.1 两个入口，两种语义

move 提供全工程唯一的两条“让机械臂动起来”的路径：

- int moveJointStep(int dir, double stepSize) —— 按关节角走一格，摇杆用这条路。返回 MOVE_NONE（参数非法未动作）/ MOVE_OK / MOVE_AT_LIMIT（到边界）/ MOVE_UNREACHABLE（超出臂展）。
- int moveToPoint(const double *goal, uint8_t seen, double maxDps, double dtSec, int mode) —— 走向一个坐标点，绘图、取放与串口坐标命令都走这条路。返回只有 MOVE_XYZ_OK / MOVE_XYZ_REJECTED 两种，只有 OK 会写 Pos。

2.3.2 moveJointStep 的五步流程

- 1) 参数检查：方向非法返回 MOVE_NONE；stepSize ≤ 0 时改用全局 speed.stepSize（于是摇杆与所有程序化动作共用同一档速度）。
- 2) 在副本上试探：SER next = Pos.ser，按方向加减一个步长。
- 3) 关节限位：clampServoAngles(&next)；夹完与原来一样（说明已经顶到行程）⇒ 返回 MOVE_AT_LIMIT。这里用的是逐字节比较 samePose()，只比 b/r/c 共 24 字节——避开三次浮点相等，理由写在代码注释里：未转动的角只有“被夹了”或“原样不动”两种可能，转动的角等于原值 ± 步长（恒大于 0），唯一“数值相等而位不同”的情况是 ±0.0，而这条路径产生不了。
- 4) 正解复核：recFromServo() 算末端，若失败 ⇒ MOVE_UNREACHABLE；用 posOutOfRange() 比对 limit，越界 ⇒ MOVE_AT_LIMIT（角度模式下坐标是派生量，所以这等价于“这个方向到头了”）。
- 5) 提交：只有全部通过才一次性写 Pos.ser = next；Pos.rec = nextRec；。任何失败路径一个字节都不写 Pos。

2.3.3 moveToPoint 的三种逼近策略

同一个“走向目标点”的需求，在三个场景下的正确行为其实不同，因此用 mode 区分：

三种模式的行为差异

mode	使用者	速度超限时	反解被限位吸附时	是否限速
------	-----	-------	----------	------

MOVE_XYZ_TRACK(0)	绘图（平移与绘制）	整点拒绝 ，本点不提交、轨迹参数不推进，调用方把位移折半重试	算失败（拒绝）	限速
MOVE_XYZ_JOG(1)	绘图示教点动	把超出部分 夹到上限 ，尽量走一点	仍算失败（宁可少走，也不原地卡住）	限速（夹取）
MOVE_XYZ_NOW(2)	串口 x/y/z 坐标命令	不适用（不限速，一次到位）	允许吸附	不限速

共同细节：goal 与 Pos.rec 同坐标系（串口的地面坐标系由串口层先减去肩高）；seen 位掩码指明哪几轴被本次请求提及，**没提及的轴沿用当前值**（x10 这类单轴命令就是这么处理的）；maxStep = maxDps × dtSec，并设 0.2° 的下限——防止 dtSec 被算成 0 时“一步都不许走”导致永远原地不动。JOG 模式下 d[i] 按 ±maxStep 夹取后提交，因此一次点动最多走这么多，超出的部分下一轮再补。

一个可复现的取舍记录：posOutOfRange() 曾经尝试复用 clampToRange() 为核心的实现（把落点放进副本、看它的返回值），move.cpp 自身从 300 B 降到 206 B——但 LTO 之后 draw_control.cpp 从 13966 B 涨到 14178 B，**全程序净增 82 B**，于是回退。这属于“局部更小、整体更大”的典型案列，说明为什么容量相关的改动必须以全程序实编为准。

代码位置：move.h:1-145（含关节角/坐标两套语义的说明）、move.cpp（223 行：DIR_CODE PROGMEM 查表、samePose、posOutOfRange、moveJointStep、六个定长函数、moveToPoint）。

2.4 路径几何核心 path_core

这是 v1.6.0 抽出来的 header-only 几何核心，四个 inline 函数构成“任何坐标路径能不能安全走出来”的唯一判据：

path_core.h 的四个函数

函数	做什么	失败意味着
pathCoreSolveJoint(x,y,z,b,r,c)	对一个点做几何检查（两级边界 + isReachable()）并反解	点不可达或超出关节行程
pathCorePointOk(x,y,z)	单点完整校验：在 limit 内 + isReachable() + 反解成功且 未被限位吸附	这个点不能安全到达
pathCoreSetJoints(b,r,c)	把解出的三个角一次性写入并刷新 Pos.rec	—（只在上面两个都通过后调用）
pathCoreSegmentOk(p0, p1, sampleStep, sampleMax, jumpDeg)	沿线段按步长采样，逐点校验，并检查相邻采样点解出的 b/r/c 是否跳变超过 jumpDeg	线段中途会掉出可达区，或反解在不同分支之间跳变

分支跳变为什么必须单独检查：两连杆在肘上/肘下两种解之间是连续的，但在“反折”分支附近切换时，两个相邻点的关节角可能相差几十度。这种点单看每一个都合法，连起来却会让机械臂“啪”地翻一下。采样校验把这类路径整条挡在启动之前（绘图与取放都因此受益）。

参数在实际调用处的取值：取放用 (0.5, 64, 25.0)，绘图用 (0.25, 96, 25.0)——步长更小、采样更多，因为绘图对笔尖轨迹的平滑度要求更高。

代码位置：path_core.h:1-105；调用点见 2.5（pick_place.cpp）与 2.8（draw_control.cpp:431-499）。

2.5 自动取放 pick_place

2.5.1 状态机与动作分解

一次取放被拆成 10 个阶段（另加空闲态），每个阶段要么“移动到某点”、要么“开合夹爪”、要么“停顿”：

取放阶段

阶段	类型	动作	参数来源
to-src-approach	移动	平移到物体正上方（飞行高度 = 台面 + 6）	PICK_APPROACH_DZ 6.0
descend-src	移动	垂直下降到台面（z = 12）	PICK_GRASP_Z 12.0
close-tool	夹爪	合爪到行程下限	PICK_TOOL_CLOSE_ANGLE = servoLimit.minF（60°）
dwel-close	停顿	合爪后停留，让夹爪真正夹稳	PICK_DWELL_MS 400
lift	移动	垂直抬起到飞行高度	PICK_APPROACH_DZ
traverse	移动	水平平移到放置点上方（保持飞行高度）	PICK_DST 表
descend-dst	移动	垂直下降到台面	PICK_GRASP_Z
open-tool	夹爪	松爪到行程上限	PICK_TOOL_OPEN_ANGLE = servoLimit.maxF（150°）
dwel-open	停顿	松爪后停留	PICK_DWELL_MS
retreat	移动	垂直撤离到飞行高度，序列结束	PICK_APPROACH_DZ

2.5.2 位置表与一个被自检强制执行的约束

三个物体的初始位置与放置位置（坐标单位与臂长一致）

物体	初始位置 (x, y)	放置位置 (x, y)	$\Delta x / \Delta y$
A	(24, 12)	(16, -14)	-8.0 / -26.0
B	(24, -12)	(18, 16)	-6.0 / +28.0
C	(12, 20)	(28, -6)	+16.0 / -26.0

三个物体的初始位置互不相同，放置位置也互不相同，且每个物体的放置位置相对自己的初始位置在 x 和 y 两个方向都有明显位移（不是原地挪一点）。取放高度统一 12.0，飞行高度 6.0。位置表放在 PROGMEM 里，省下 72 B SRAM。

被自检强制执行的约束：“放置点不能离别人的初始点太近”——否则机械臂放下物体时可能撞到旁边那个。自检探针 probe_pick_place 直接对这个表做断言，实测最小距离 7.2111（ $\sqrt{52}$ ，三对并列）。初选的两组参数（B 放（14, 16）、C 放（26, -8））因为距离只有 4.47 而被改掉。这条约束是“由测试固化设计意图”的例子：改表就会被自检拦住。

2.5.3 为什么走直线，以及启动前的整条校验

取放的每个移动阶段都在 x/y/z 上做线性插值（所以下降和抬起是垂直的，平移是水平的），插值点交给 getAngleEx() 反解成关节角。为了让“整条路径安全”在动作开始前就有结论，pickPlaceStart() 会先把整条路径采样验证一遍：

- 1) 每个采样点在 limit 内且 isReachable()；
- 2) 反解出的关节角没有被限位吸附（吸附说明这个点其实到不了）；
- 3) 相邻采样点的反解结果不跳分支（b/r/c 变化不超过 25°）；
- 4) 启动时手里的关节角与“当前坐标反解出的角”相差不超过 10°（PICK_START_TOL_DEG）——否则说明姿态落在另一个分支上，第一段会猛跳，宁可拒绝启动。

任一条不满足，pickPlaceStart() 返回 -3 并一个字节都不改；返回码完整含义：0 已开始 / -1 物体编号非法 / -2 已有序列在执行（本次忽略）/ -3 路径校验失败。

速度与档位联动：移动阶段的速度从 25 / 50 / 90 度每秒（慢/中/快）中按当前档位选取，夹爪阶段固定 60 度每秒；因为每轮只走“速度 × 本轮时长”，所以取放进行中调档位会立刻生效。每阶段的时长被夹在 PICK_MIN_SEG_MS 180 ~ PICK_MAX_SEG_MS 4000 之间，避免“一步跨太远”或“一步走太久”。

2.6 摇杆运动控制 joystick_control

2.6.1 四路映射与方向语义

摇杆到关节的映射（四路可同时动作）

摇杆	引脚	关节	推的方向与效果
左手柄 左右	A0	基座 b (angle1)	角度增大方向（末端往 +y 侧转）
左手柄 前后	A1	上臂 r (angle2)	前推 = 角度减小，后拉 = 角度增大（上机实测后调转）
右手柄 前后	A3	下臂 c (angle3)	前推 = 角度减小，后拉 = 角度增大（上机实测后调转）
右手柄 左右	A2	夹爪 f (angle4)	角度增大 = 张开

四路各自独立时间门控、独立走后一个格。**顺序是承重的：**同一轮里若有两路都到了该走的时间，按 base → shoulder → elbow → tool 的顺序依次执行，先动的那一路会改变后一路的越界校验基准。方向编码（JOY_DIR_*）刻意与 move.h 的 MoveDir 保持一致的数值关系，使两者可以互传而不用查表转换。

2.6.2 中位自标定与死区

摇杆的机械中位并不精确等于 ADC 中点 512，且个体差异明显。固件在 joystickSetup() 里对四路各采 8 次求平均作为中位（JOY_CAL_SAMPLES 8）——**必须在机械臂动作之前做**，因为那时没人碰摇杆，采样最干净。若某路的中位偏离 512 超过 64（JOY_CAL_MAX_OFF），说明读数不像“静止的摇杆”，直接退回 0，宁可用理想中点也不用可疑值。偏转小于 JOY_DEADZONE 15 视为没有推杆，避免抖动导致机械臂缓慢漂移。

2.6.3 速度曲线：偏转越大越慢

speedIntervalMs(mag) 把偏转幅度映射为“两次步进之间的毫秒间隔”：`interval = minDelay + mag/500 × (fullDelay - minDelay)`。其中 minDelay 是最快间隔（轻微偏转时连续快走），fullDelay 是最慢间隔（满偏时最慢最稳）。这条“推得越狠越慢”的约定同时被摇杆、绘图示教点动与（作为上限）绘制使用，属于全工程统一的“慢而稳”原则。

2.6.4 让位规则

- joystickLoop() 里一行决定让位：`locked = pickPlaceIsBusy() | buttonControlLocked() | drawControlLocked();`。用按位或（而不是逻辑或）是为了省掉短路分支。三处细节：
- **buttonControlLocked() 刻意不包含“正在录制”**——录制的目的就是记录操作者的摇杆动作，此时摇杆必须活着。
 - **五点示教时摇杆被 drawControlLocked() 挡住**，改由绘图模块自己读摇杆做“笛卡尔点动”（见 2.8.5）：此时摇杆推动的是笔尖坐标而不是关节角。
 - **取放序列期间摇杆被让位**，序列独占 b/r/c 三个关节。

2.6.5 为省空间做的寄存器级操作

本模块有两处刻意“不用 Arduino API”的地方，都是为了在 600 B 余量下生存：初始化直接写 DDRC/DDR0/PORTB 寄存器（pinMode/digitalWrite 是非内联库函数，五次调用要多花三十多字节）；

LED 指示直接写 PORTB5; lastStepTime[i] = millis() 刻意写在循环体内而不提到循环外——否则 GCC 会把数组访问完全展开成 16 条 sts 指令，代码从 58 B 涨到 100 B。

代码位置： joystick_control.h:1-133（映射与操作说明）、joystick_control.cpp（477 行；speedIntervalMs :272、toolStep :292、updateLed :316、joystickSetup :328、joystickLoop :367、joyCalibrateAxis :181、readAxisAmp :195、joystickReadState :206）。

2.7 按键与录制回放 button_control

2.7.1 四个动作与三种触发通道

本模块实现四个动作：**循环取放**（按顺序对 A→B→C 发起取放）、**录制**（第一次触发开始，再次触发结束并保存）、**播放**（重放上一次录制）、**回中**（回到开机待机位姿）。四个动作各有两个触发通道，效果完全相同：

四动作与触发方式

动作	物理按键（默认关闭）	串口命令	备注
循环取放	D2	N	每次触发对下一个物体发起取放
录制	D3	R	第二次触发结束并保存；数据不合格则丢弃（回 DISCARD）
播放	D4	P	录制中直接发 P 会先收尾再播放
回中	D5	M 或 0	回到开机待机位姿，带 1500 ms 平滑过渡

2.7.2 录制：为什么每条 4 字节、为什么 200 ms

录制缓冲是静态 SRAM：每 WEARM_REC_TICK_MS（200 ms）写恰好一条 4 字节记录，四个关节各占 8 位有符号增量，单位 0.5°（BTN_ANGLE_UNIT 2.0 表示 1° = 2 单位）。于是：缓冲占用 = 条数 × 4 = 768 B，单次录制上限 = 条数 × 周期 = 38.4 秒。

早期版本是 128 条 × 3 字节（每关节 6 位字段）。问题在于 6 位字段只能表达 ±16°（32 单位 × 0.5°），而 200 ms 一条意味着 80°/s 就顶格——摇杆满偏录十几秒，几乎每条都饱和，回放终点自然对不上。改成 8 位后每条约 ±64°（317°/s），正常档（100°/s）与中速档都录得下。**更快的极端动作（快速档 400°/s）也不会丢角度：**超出字段的部分会被攒进随后的记录

（btnRecCollect() 里的进位变量），收尾时再连续补写几条把尾巴走完（btnStopRecording()）；代价是时间轴被压扁（回放比实际快）。这个取舍在文档里明说，而不是假装无损。

合格判据：结束录制时要求末端总位移不小于 BTN_REC_MIN_TRAVEL 10.0；不满足就整条丢弃并回 DISCARD。**不再要求最短录制时长——**“必须录满 10 秒”的判定在 v1.4.0 被删除（省 58 B，也更符合直觉：录多久是操作者的事）。

2.7.3 播放：4 个 50 ms 子步插值

播放时把每条 200 ms 的记录再拆成 4 个 50 ms 子步（BTN_PLAY_SUBSTEPS 4）线性插值输出，笔直跟随的观感明显更连续；把它设为 1 就退回“整条一次跳完”。播放与回中都先经过 1500 ms 的平滑过渡（BTN_RAMP_MS），避免从当前姿态猛地跳到轨迹起点。

2.7.4 让位与互斥

- **录制中：**摇杆照常可用（录制就是要录它）；串口 O/S/k/K 与 x/y/z 被拒绝；取放序列无法启动。
- **播放/回中中：**摇杆让位（buttonControlLocked() 为真）；串口动作命令被拒；只有 H/L/1/2/3 调速有效。
- **取放进行中：**四个动作全部被拒（回 BUSY）。

代码位置: button_control.h:1-137、button_control.cpp (782 行)。关键函数:
btnRecCollect :238、btnStartRecording :259、btnRecTick :286、btnStopRecording :311 (含
收尾补写)、btnStartRamp :385、btnRampTick :395、btnPlayLoad :434、btnPlaySub :452、
btnPlayTick :462、btnActionCycle :495、btnActionRecord :529、btnActionPlay :543、
btnActionHome :577、buttonLoop :690。

2.8 绘图 draw_control

2.8.1 任务表与阶段机

当前分支提供 4 种绘图任务 (另有 2 种已删除, 见 7.4) :

绘图任务表

任务号	名称	路径来源	点数
0	直线	内置图形表 (归一化两点)	2
1	字母 V	内置图形表 (归一化三点)	3
2	五点示教折线	操作者示教 5 点, 逐段直线连接	5
3	五点示教曲线	操作者示教 5 点, 向心 Catmull-Rom 平滑	5

一次绘制的阶段流转是: 待机 → 回待机 (关节空间插值) → 抬笔平移到起点上方 → 落笔 → 沿轨迹绘制
→ 抬笔 → 回待机; 示教任务在前面插入 示教中 → 停顿 (5 点记完) 两段。每个阶段每轮循环推进一步, 因此绘制全程可响应串口、可暂停、可取消, 也不会阻塞摇杆采样。

2.8.2 抬笔高度与“画不了就降一点”

抬笔高度取“落笔点高度 + 4.0”。但机械臂在靠近工作空间边缘时可能抬不了那么高 (反解不可达), 于是 liftZFor() 从 +4.0 开始按每次 0.5 向下试探, 直到找到一个可达的抬笔高度; 如果连纸面高度都不可达, 返回纸面高度并让路径校验失败。这使“边缘区域也能画, 只是抬笔低一点”, 而不是直接拒绝。

2.8.3 纸面标定

画在哪里由三个可标定参数决定, 都可用串口现场修改, 无需重新编译:

纸面标定参数

参数	默认	允许范围	串口命令
纸面高度 (笔尖落在纸上时的 z)	12.0	-5.0 ~ 30.0	p<值>, 如 p12.5
图形半宽	6.0	2.0 ~ 12.0	n<值>, 如 n6
图形中心 (x, y)	(20, 0)	每轴 坐标 ≤ 35	o<x>,<y>, 如 o20,0

内置图形的顶点用归一化坐标 $u, v \in [-1, 1]$ 存储 (直线是 $(-1, 0)-(1, 0)$, 字母 V 是 $(-1, 1)-(0, -1)-(1, 1)$), 画的时候按“中心 + 半宽 $\times (u, v)$ ”展开。默认区域覆盖 $x \in [14, 26]$ 、 $y \in [-6, 6]$, 最远的点半径约 29.6, 远小于两连杆总长 40, 属于工作空间内非常安全的一小块。

2.8.4 速度规划与“一次最多走 0.02”

绘制时笔尖线速度上限 3 单位/秒、加速度上限 6 单位/秒², 速度按“梯形曲线”生成: $v_{End} = \sqrt{2 \times \text{加速度} \times \text{剩余距离}}$ (表示“现在开始刹车刚好够”) 并夹到上限; $v_{Ramp} = v_{Cur} + \text{加速度} \times dt$; 取两者较小值。因此起步加速、中段匀速、终点减速停住, 不需要预先计算速度表。折线拐角不做特意减速, 靠 3 单位/秒的温和速度过弯。

在此基础上还有一层“单次写入位移上限 0.02 单位” (DRAW_STEP_MAX): 无论速度规划算出多大的步, 一次写入的位移都不超过 0.02 工作区单位 (一条 12 单位长的直线至少分 600 步)。这样每一

小步的反解误差都不会被放大，笔尖轨迹更稳。这里有一个被实测否决的更“自然”的写法：把这一轮拆成多个 0.02 的小步循环走。实测在做 LTO 优化时，循环导致 `pathAdvance()` 无法内联，flash 多花 662 B，默认配置直接涨到 32336 B——超过 Uno 可用的 32256 B，根本编译不进去；而“这一轮最多走 0.02、剩下的下一轮补”只多花 32 B，效果相同（任何一次写入位移都不超过 0.02）。最终采用后者。

2.8.5 示教点动：推的是笔尖，不是关节角

五点示教时，摇杆改为控制**笔尖坐标**：左手柄左右推 → x，左手柄前后推 → y（前推 y 增大），右手柄前后推 → z（前推往下压）；右手柄左右（原本是夹爪）默认不响应，因为夹爪里夹着笔。这么做的理由很直接：绘制任务的目标是“把笔尖移到纸面上的某个点”，用关节角去推根本对不准（一个关节角变化会同时改变 x、y、z）。

每格的位移按偏转比例缩放：轻微偏转 0.02 单位、满偏 0.25 单位（约 6 单位/秒，比自动绘制快一倍，因为手动操作要的是“别卡”）；步进间隔在最快的 `minDelayMs` 到最慢的 `fullDelayMs` 之间插值，且**偏转越大间隔越长**（早期版本写反了，表现为“推得越狠冲得越快”）；因此示教点动也吃 H/L/1/2/3 档位。**注意这三路的步长不是 `speed.stepSize`**——那个量是“关节角每周期转多少度”，曾被误当作位移用，结果笔尖一格跳出半个工作区；现在两者彻底分开。

2.8.6 启动前的四条校验

`validatePath()` 在开始绘制之前跑完，任一条不合格就拒绝启动（回 REJECTED），Pos 一个字节不改：

- 1) 点数至少 2；
- 2) 从待机位到起点正上方这一段（抬笔平移）按 0.25 步长、最多 96 个采样点校验；
- 3) 垂直落笔那一段同样校验；
- 4) 轨迹本体：折线逐段采样校验，曲线则每段取 24 个参数采样点逐点校验，并要求相邻采样点的 $b/r/c$ 变化不超过 25° （分支不跳变）；
- 5) 终点抬笔高度用 `liftZFor()` 求，再校验一次。

2.8.7 暂停、继续、取消

Q 暂停：停在原地，任务状态全部保留。U 继续：从暂停处接着走。W 取消：先抬笔再回待机。取消这里有一个必须显式处理的细节：取消时如果只清标志而不把阶段改成“抬笔”，状态机仍停在

TRAVEL/PLUNGE/PATH 上，下一轮会把这一笔接着画完——“取消等于没取消”。因此 `beginLiftToReturn()` 会同时设置阶段为 LIFT、并记录“抬笔完成后去 RETURN”。

2.8.8 卡死保护

`stallGuard()` 统计“连续多少轮没有产生实际位移”：只要有一次位移超过 $1e-6$ 就清零，连续 500 轮（`DRAW_STALL_LIMIT`）都没有位移就放弃本次任务，置回待机并把结果码记为 REJECTED。这避免了在某种边界条件下“循环空转、既不走完也不报错”的悬挂状态。

代码位置：`draw_control.h`:1-274、`draw_control.cpp` (1388 行)。参数集中在 :66-124；`speedLimit` :539、`applyDtSec` :509、`applyPointStep` :528、`stallGuard` :556、`pathAdvance` :715、`pathTick` :750、`validatePath` :431、`liftZFor` :419、`curveSpan` :308、`jogIntervalMs` :809、`teachJogTick` :825、`drawLoop` :1265。

2.9 串口协议 `serial_protocol` + `protocol_constants`

2.9.1 命令表

串口命令全表（大小写区分）

命令	含义	回复
O / S	夹爪张开 / 闭合并走到行程两端	OK (附当前夹爪角度: OK 90°)
H / L	整体速度提升 / 降低一档	OK
1 / 2 / 3	直接切到慢 / 中 / 快档 (v0.2.0 起的老命令)	OK
k / K	兼容老命令: 夹爪步进张开 / 闭合 5°	OK (附角度)
x<值>,y<值>,z<值>	末端坐标运动 (可只写一部分轴, 未写轴保持不动)	OK / REJECTED / BUSY
A / B / C	对物体 A/B/C 启动自动取放序列	OK / BUSY / REJECTED
N / R / P / M / Ø	四个动作的串口孪生: 循环取放 / 录制 / 播放 / 回中	OK / BUSY / OFF / DISCARD / EMPTY
F / D / G / E / Q / U / W	绘图: 选任务 / 开始 / 记点 / 撤销 / 暂停 / 继续 / 取消	OK / BUSY / REJECTED / OFF
p<值> / n<值> / o<x>,<y>	绘图标定: 纸面高度 / 半宽 / 中心	OK / REJECTED
!	查询当前编译且启用的功能	P=1 B=1 D=1 形式
!P / !B / !D	运行时切换取放 / 按键 / 绘图 (未编译的功能不能开)	OK / REJECTED

每条命令都会回一行短回复，这点是 v1.2.0 修的一个真实缺陷：早期版本里很多“已被接受的命令”执行完什么也不打印，终端看起来像卡死，用户无法区分“命令生效了”和“命令没被识别”。现在回复语只有 7 种，含义如下：

回复语

回复	含义
OK	命令已执行 (O/S/k 附带夹爪角度)
BUSY	命令看懂了，但机械臂正忙 (取放/录制回放/绘图进行中)
REJECTED	这台固件没编译该模块；或坐标不可达/在地面以下；或绘图路径校验不过
OFF	固件里有该模块，但被 !P/!B/!D 关掉了
DISCARD	录制结束但数据不合格 (没位移/缓冲满)，已丢弃
EMPTY	播放时还没有录制数据
ERR	语法错误 (坐标写坏，或给单字符命令加了尾巴)

2.9.2 坐标系转换与“地面以下”判定

用户看到的坐标原点在“过肩关节的地面垂足”，而固件内部坐标原点在肩关节，两者只差一个肩高 WEARM_SHOULDER_HEIGHT (默认 20.0，定义在 serial_protocol.h:50-52，是全工程唯一真值)。于是串口层在解算前执行 $z_{\text{内部}} = z_{\text{坐标}} - 20.0$ 。默认肩高 20 时：x20,y0,z40 正是开机初始位姿；可达范围 $x, y \in \pm 40, z \in [0, 60]$ ；z 为负 (地面以下) 直接回 REJECTED。这个约定还有个好处：实机标定时只需量一下肩关节离地多高、改这一个数即可。

2.9.3 运行时的功能开关

编译开关决定“哪些功能在固件里”，运行时开关决定“在固件里的功能此刻是否可用”。固件持有一个运行时功能位图 s_runtimeFeatures，初值由编译期开关算出，在 serialProtocolBegin() 时复位。! 查询输出形如 P=1 B=1 D=1; !P / !B / !D 切换对应功能。门控只作用于串口分发：不影响实体按键，也不影响已经在运行的任务 (比如绘图正画着，!D 关掉不会让它停下)。对未编译的功能发切换命令一律回 REJECTED，因此“关掉再打开”不可能凭空获得一个没编进固件的模块。

2.9.4 忙守卫的例外

机械臂忙时（取放/录制回放/绘图进行中），动作类命令一律回 BUSY；但有一类命令必须永远有效：**调速命令**（H/L/1/2/3）。理由是安全——如果序列跑得太快，操作者需要能立刻踩刹车。另外！查询与绘图/按键命令按各自模块的让位规则处理。

2.9.5 单读者与裸 UART 后端

- **单读者：**串口字节只准由 serialProtocolLoop() 读取。历史上同时存在 handleSerialSpeedCmd() 与协议模块两个读者，两个人各吃掉一半数据，行缓冲永远拼不出完整指令——这个函数已被删除，头文件里明确写了“不要再同时调用两者”。
- **行缓冲 40 字节 + 300 ms 静默超时：**超时兼容“串口助手没勾发送新行”的场景——一行迟迟收不到结束符时，静默 300 ms 就按“整行已到”处理。
- **裸 UART 后端：**默认直接写 UDR0/UCSR0A，接收走 USART RX 中断 + 64 字节环形缓冲（省 852 B flash / 109 B SRAM）；定义 WEARM_SERIAL_ARDUINO=1 可退回官方 HardwareSerial 后端以便排查。
- **回复层刻意不用 Serial.print() 打数字：**那会把 Arduino 的数字/浮点格式化层（约 9.5 KB）拖进镜像；回复走 protoPut()（裸写 UDR0），夹爪角度只输出整数。
- **没有独立的 help 命令：**命令表只在 serialProtocolBegin() 的启动横幅里打印一次。

代码位置：protocol_constants.h:1-103（命令字符、轴映射、行缓冲与波特率、档位边界）、serial_protocol.h:1-128（命令表说明、回复语义、坐标系、28 个结果码 PROTO_RES_*、三个对外函数）、serial_protocol.cpp（973 行）。

2.10 单文件整合版

除模块化源码外，仓库还提供一个“全工程合并成单个 .ino”的产物，供只能提交单个文件的场合使用（如某些课程作业平台）：single/meArm/meArm.ino，由 docs 同级的生成器 .selfcheck/make_single.ps1 生成，内容 332015 字节。

单文件整合的关键约定

事项	处理方式
拼接顺序	11 个头文件（配置 → 几何 → 协议常量 → 舵机 → 运动 → 串口 → 取放 → 按键 → 绘图 → 摇杆 → 路径核心）→ 9 个实现文件 → 主 sketch
本地 include	全部去掉；系统头只保留首次出现；每段前加 // ===== <文件名> ===== 标记
必须先声明后使用	编译脚本不做自动函数原型插入，因此拼接顺序即依赖顺序
同一编译单元内的重名	clampDouble 同时存在于几何核心与绘图模块，绘图那一份改名 drawClampDouble；BTN_DEBOUNCE_MS（按键）与摇杆按键消抖宏同名，摇杆那份改名 JOY_BTN_DEBOUNCE_MS
合法的重名	beginMove 在两处签名不同（一处三坐标、一处阶段+数组），构成合法重载，无需改名
换行符	生成器显式写 LF，保证“工作区文件 = 检出文件 = 入库 blob”
体积差异	单文件版比模块化版多 20 B（31676 B vs 31656 B），来自 LTO 内联差异（main +24 B、getAngleEx +8 B、clampServoAngles -20 B），功能完全相同

生成器是**幂等的**：重复运行得到的文件逐字节相同（SHA256 一致），且可以从干净的 HEAD 重新生成（不依赖工作区的未提交改动）。单文件版不参与固件本体编译，只在需要时由脚本生成。

3 专题：示教点记录

本固件里“示教”不是一个功能，而是三条并行的通道：它们解决的需求不同，因此数据表示、存储方式和结束条件都不同。这张表先把三者放在一起对比，后面每小节展开。

3.1 三条示教通道总览

三条通道对比

	通道一：摇杆直接示教	通道二：按键录制	通道三：绘图五点示教
记的是什么	不记录（实时动作）	关节角增量序列（相对上一条）	笔尖坐标点（绝对 x/y/z）
触发方式	推摇杆即动	串口 R / 按键 D3：再触发一次结束	串口 D 进入示教，G/E 记点与撤销
存储	无	静态 SRAM 768 B（192 条 × 4 B）	RAM 数组：5 点 × 3 双精度
上限	—	38.4 秒（192 × 200 ms）	5 个点
结束条件	松手即停	再次触发 R，且末端总位移 ≥ 10.0	记满 5 点；或 ≥ 2 点后手动结束；或停顿 700 ms 自动开始
精度损失	无	有损：0.5° 量化，极快动作会压扁时间轴	无：点为原始坐标值
校验	每步越界即停住	结束时不合格整条丢弃（DISCARD）	启动前整条路径采样校验，不通过拒绝启动
之后的用途	随手摆放、对准	一键重放同一段动作	生成折线或曲线并自动绘制

3.2 通道三：绘图五点示教（最完整的一条）

3.2.1 为什么要改成“点笔尖”

最初版本的示教是“推摇杆改关节角，记下关节角”。问题在实际使用时立刻暴露：绘制任务要的是“把笔尖移到纸面上的目标点”，而一个关节角的变化会同时改变笔尖的 x、y、z——操作者永远对不准。因此示教模式下摇杆改为控制笛卡尔坐标：左手柄左右 → x，左手柄前后 → y（前推 y 增大），右手柄前后 → z（前推往下压，因为“压到纸上”的直觉就是往下），右手柄左右（本来是夹爪）默认不响应，以免把夹着笔的夹爪打开。

3.2.2 数据与流程

五点示教的数据结构

变量/数组	含义	用途
s_pts[5][3]	最多 5 个点，每点 x/y/z	轨迹的控制点
s_ptCount	当前点数	判断能否结束（≥2）与是否已满（5）
s_ptArc[5]	每个控制点处的累计弧长	生成后用于“按弧长推进”与“过点统计”
s_ptHit	位图：哪几个点已被笔尖经过	绘制完成后的结果统计（命中几个点）

完整流程：① 发 D（或按键）启动示教任务，机械臂先在关节空间插值回到待机位（这样每次示教都从同一个姿态开始，点动不会一上来就顶到限位）；② 进入示教阶段，摇杆按上面的笛卡尔映射点动笔尖，每格的位移与间隔见 2.8.5；③ 每按一次 G（或按键 1）记录当前笔尖坐标，点满 5 个自动进入“准备绘制”；④ 每按一次 E（或按键 2）撤销最后一个点；⑤ 记满 5 点后停顿 700 ms

（DRAW_TEACH_START_DELAY_MS，留出“我还想再检查一下”的时间）自动开始；也可以用按键 4 / drawTeachFinish() 提前结束（此时至少要有 2 个点）；⑥ 结束时调用 validatePath() 整条校验，不通过就拒绝启动并保持示教结果不变。

为什么是 5 个点：曲线需要至少 4 个控制点才有“中间段”（3 点只能得到两段直线式的圆滑连接），而手动点动一个点需要几十秒；5 点兼顾“画得出来”与“教得完”，同时也是 SRAM 友好（ 5×3 双精度 = 120 B）。上限而非动态长度的原因很简单：静态数组不需要堆，也不会有“录到一半没内存”的失败路径。

同一次示教同时支持两种画法：任务 2（折线）把 5 个点顺次用直线连起来；任务 3（曲线）把同样的 5 个点当作 Catmull-Rom 控制点做平滑。也就是说操作者只需要教一次点，之后可以在两种画法之间切换（F 命令循环选任务）。

3.3 通道二：按键录制的数据布局

3.3.1 一条记录 4 个字节

录制记录的字节布局

字节	内容	取值范围
1	基座 b 的角度增量	-128 ~ +127 个单位
2	上臂 r 的角度增量	同上
3	下臂 c 的角度增量	同上
4	夹爪 f 的角度增量	同上

单位是 0.5° （BTN_ANGLE_UNIT 2.0，即 $1^\circ = 2$ 个单位），因此单条记录每个关节能表达 $\pm 64^\circ$ 的变化。**存的是增量而不是绝对值：**增量域更小（8 位足够），且播放时从当前姿态出发逐步累加，天然与“当前在哪里”无关——不必先回中再播放。四个关节各占一字节、都按有符号处理，所以 192 条正好 768 B，全部是静态 SRAM，不涉及动态分配。

3.3.2 采样与进位：不丢角度的机制

每 200 ms 触发一次采样（btnRecTick()），采样的瞬间计算四个关节相对上一条记录的增量，然后依次执行：

- 1) 把增量按 0.5° 单位换算成整数（btnToUnits()，带四舍五入）；
- 2) 与上一次“没能写进去的余量”相加（进位变量），得到本次实际要写的值；
- 3) 若某关节的值超出 -128~127 域，只写它能表达的部分（夹到边界），**剩余部分留在进位变量里**，下一次采样时继续带上；
- 4) 四个关节的四字节记录写入缓冲区并递增写指针。

这套进位机制的效果是“**角度不丢、时间轴可能被压扁**”：以最快档（ $2^\circ / 5\text{ ms} = 400^\circ / \text{s}$ ）猛推摇杆时，单条记录装不下 200 ms 内的 80° 变化，多出来的部分被后面的记录依次消化；录制结束时 btnStopRecording() 还会连续补写若干条，把尾巴走完。回放因此看起来比实际操作“快一点”，但**终点姿态对得上**——这正是当初从 6 位字段（ $\pm 16^\circ$ ，几乎必然顶格）改成 8 位字段的原因。

3.3.3 合格判据

结束录制时要求末端总位移不小于 10.0（BTN_REC_MIN_TRAVEL），这里的“位移”是整段轨迹三轴包围盒的最大跨度（而不是起终点直线距离），因此“来回大幅摆动但回到原点”也算有效录制。不满足则整条丢弃并回 DISCARD。缓冲写满也是同样的处理（丢弃，并提示）。

3.4 三条通道为什么不合一

把三者统一成“都存坐标点”或“都存关节角”都会付出代价：

- **坐标 vs 角度：**录制通道要复现的是“人手推出来的关节动作”，它可能包含关节来回摆动、夹爪开合等与末端坐标无关的运动；用坐标表示会丢掉 angle4 的信息（反解不含 angle4）。反过来，绘图示教的目标是纸面轨迹，用角度表示会让操作者无法对准。

- **连续序列 vs 少量控制点**：录制是长时间、等间隔的连续序列，适合有损压缩与固定速率回放；示教是少量、不等间隔的精确控制点，适合原值保存与弧长参数化。
- **实时 vs 离线**：摇杆通道必须在每个控制周期都能走一小格（不能等“记录完”），因此它不存储、不校验，只做单步越界判断；另两条通道都可以在开始前做完整校验。

4 专题：轨迹生成

4.1 四条来源，一个出口

固件里所有“机械臂自己沿着一条路走”的行为都归一到同一个出口：`moveToPoint()`（见 2.3.3）。区别只在于**轨迹怎么生成、按什么参数推进**。

四条轨迹来源

来源	几何定义	参数化	推进方式
内置图形（直线/字母 V）	归一化顶点表 → 纸面坐标	折线：累计段长	按弧长推进，段内线性插值
五点示教折线	5 个示教点顺次连线	折线：累计段长	同上
五点示教曲线	5 个控制点的向心 Catmull-Rom	曲线：弧长表（每段 64 采样）	按弧长推进，段内反解参数 u
取放（A/B/C）	物体上方/台面/放置点上方的折线路径	阶段序列：每段独立线性插值	每段按速度 $\times$ 时长推进，走完换下一阶段
录制回放	录制得到的角度增量序列	时间：200 ms/条再分 4 个 50 ms 子步	按时间推进，子步内线性插值

4.2 折线轨迹

折线是“顶点 + 直线连接”，它的参数化就是弧长：`polyMeasure()` 逐段用两点距离累加得到总长，`polyPointAt(s)` 根据累计段长定位到某一段并做线性插值（插值参数夹在 $[0, 1]$ ）。为了让“按弧长推进”和“按剩余量刹车”都能工作，折线只需要“总长”与“当前已走弧长”两个标量，不需要为每个采样点存数组——这也解释了早期版本里被删掉的 `s_segLen[]`：重算的结果与查表逐位相同，省下的是 RAM。

4.3 曲线轨迹：向心 Catmull-Rom

4.3.1 为什么选它

需求是“经过给定的 5 个点、并且看起来圆滑”。Catmull-Rom 样条的优点正是**曲线严格经过控制点**（对“示教的点就是纸上的点”这个语义至关重要），而 B 样条会偏离控制点。选**向心（centripetal）**参数化而不是均匀参数化，是因为均匀参数化在手写示教点间距不均时会出现自交打圈或尖角；向心参数化用点间距离的平方根作为节点间距，能消除这两类伪影。

4.3.2 计算方式

对第 i 段（控制点 P_i 到 P_{i+1} ），先算节点间距 $dt = |P_{i+1} - P_i|$ 的平方根（`knotDelta()`；若距离小于 $1e-6$ 则取 0.001，避免除零）。两个端点的切线分别由相邻点差商给出：

- 起点切线： $t1 = (P_1 - P_0) / dt \times dt$ ；
- 终点切线： $t2 = (P_{n-2} - P_{n-3}) / dt \times dt$ （**不外推**，因此曲线不会冲出最后一个点）；
- 内部点： $t1 = (P_{i+1} - P_{i-1}) / (knotDelta(i-1,i) + dt) \times dt$ ， $t2 = (P_{i+2} - P_i) / (dt + knotDelta(i+1,i+2)) \times dt$ 。

段内位置由三次 Hermite 基函数给出（ $u \in [0, 1]$ ）：位置用 `h00/h10/h01/h11` 四个基函数，切线用 `g10/g11` 的导数形式 `e0/e1`；实现上位置与切线共用一个循环（拆开实测多花 84 B flash，把切线抽成独立函数多花 170 B）。这里还利用了一个数值细节：`e1 = -e0`，对 IEEE 双精度而言“取负”是精确的位操作，因此少做一次乘法（对没有硬件浮点的 AVR 来说，每一次软浮点调用都值得省）。

4.3.3 弧长参数化

曲线按参数 u 推进会出现“参数走得快的地方笔尖跑得慢”的问题，导致笔速不均匀，因此固件对曲线做弧长参数化：`curveMeasure()` 对每一段取 64 个采样点累加实际距离，得到每段的长度与每个控制点处的累计弧长。运行时按“目标弧长”反推该段内的参数 u ($du = \text{步长} / \text{段内速度}$)，从而做到“不论曲线弯直，笔尖线速度都等于设定值”。

过点统计：每走过一个控制点的弧长后，测量笔尖与那个点的实际距离，若不超过 0.8 (DRAW_POINT_TOL，即“肉眼认为画到了”的容差) 就记为命中一次。绘制结束后可以通过 `drawLastPointsHit()/drawLastPointsTotal()` 读出“5 个点命中几个”，作为一次绘制质量的可核查指标。

4.4 内置图形怎么变成坐标

内置图形存的是归一化顶点 ($u, v \in [-1, 1]$)：直线是 $(-1, 0)$ 到 $(1, 0)$ ，字母 V 是 $(-1, 1) \rightarrow (0, -1) \rightarrow (1, 1)$ (左上 $\rightarrow$ 底尖 $\rightarrow$ 右上，一笔画完的顺序)。画的时候展开成纸面坐标： $x = \text{中心} x + u \times \text{半宽}$ 、 $y = \text{中心} y + v \times \text{半宽}$ 、 $z = \text{纸面高度}$ 。这样的好处是“改大小只改一个数(半宽)、改位置只改两个数(中心)”，而且可以从串口现场标定 (n/o 命令)，不需要重新编译。

4.5 取放路径的形状

取放路径不是“从起点到终点的直线”，而是一条**分阶段折线**，每个阶段承担一个明确的安全职责：水平移动永远发生在“飞行高度”上，垂直移动永远发生在物体/放置点的正上方。因此物体的位移过程中不会出现“斜着刮过台面”或“撞到旁边的物体”。路径的几何定义见 2.5.2 的位置表；每段的推进方式是“速度 $\times$ 本轮时长”，段内线性插值；每段时长被夹在 180 ~ 4000 ms 之间。走完一段进入下一阶段，整条序列不需要预先算出采样点数组（只在启动校验时临时采样）。

4.6 录制回放轨迹的插值

回放是**按时间**推进的：每条记录代表 200 ms 的目标增量，播放时把它拆成 4 个 50 ms 子步 (BTN_PLAY_SUBSTEPS)，每个子步走 1/4 的增量。这样 5 Hz 的采样率在观感上变成 20 Hz 的平滑运动；把子步数改成 1 就退回“每 200 ms 跳一次”的原始效果。播放（以及回中）之前先做 1500 ms 的平滑过渡 (BTN_RAMP_MS)，避免从当前姿态突然跳到轨迹起点造成冲击。

4.7 完整校验链

轨迹的正确性由三道独立的关卡保证，任何一道不通过都不会开始动作：

轨迹校验链

环节	检查内容	不通过时
启动前（取放/绘图）	整条路径采样：在坐标边界内、可达、反解未被限位吸附、相邻采样不跳分支 ($\leq 25^\circ$)	拒绝启动，回 REJECTED，姿态一字节不改
启动时（取放）	当前关节角与“当前坐标反解角”的差 $\leq 10^\circ$	拒绝启动（防止落在另一个分支导致第一段猛跳）
运行时（每一步）	MOVE_XYZ_TRACK 模式下任一关节需要的变化超过 速度 $\times$ 时长 就整点丢弃	本点不提交、轨迹参数不推进、位移折半重试（最多 4 次）
运行时（绘图）	连续 500 轮没有产生实际位移	放弃本次任务并回待机（防悬挂）

5 专题：速度控制

5.1 三层结构

速度在本固件里不是“一个数字”，而是三层叠加的结果。粗调由操作者决定（档位），细调由各个动作的规划器决定（每轮走多少），最后还有一道与安全相关的硬上限（关节角速率）。

- 1) **第一层：档位。**三个预置档位（慢/中/快），每个档位同时定义三个量：每格转角、最快间隔、最慢间隔。切换方式：串口 H（升档）/L（降档）/1/2/3（直接指定）。取放序列进行中调档位也会立即生效。
- 2) **第二层：各动作的速度规划。**摇杆按偏转量在最快与最慢间隔之间插值；绘图按梯形速度曲线（加速—匀速—减速）计算笔尖线速度，并保证单次写入位移不超过 0.02 单位；示教点动按偏转量在 0.02 ~ 0.25 单位之间缩放步长；取放按档位查 25/50/90 度每秒。
- 3) **第三层：关节角速率上限。**绘图对关节速率设了 $70^\circ/\text{s}$ 的硬上限（DRAW_MAX_DPS）：若某一步反解出的角度变化超过它，就把这一步的位移折半重试（最多 4 次）；仍超限则本轮不走、轨迹参数也不推进。它的作用是“保护机械结构”，因为在工作空间边缘，笔尖走很短的距离也可能要求关节快速转动。

5.2 三个档位的精确参数

档位表（中速为出厂默认）

档位	每格转角 stepSize	最快间隔 minDelayMs	最慢间隔 fullDelayMs	最快角速度（估算）
慢速（SPEED_SLOW=0）	0.5°	20 ms	80 ms	$\approx 25^\circ/\text{s}$
中速（SPEED_NORMAL=1，默认）	1.0°	10 ms	40 ms	$\approx 100^\circ/\text{s}$
快速（SPEED_FAST=2）	2.0°	5 ms	20 ms	$\approx 400^\circ/\text{s}$

三个档位之间是精确的 2 的幂关系：步长 $0.5 \times 2^{\text{档位}}$ 、间隔 $20 \gg \text{档位}$ 与 $80 \gg \text{档位}$ 。因此实现上不需要三条 `setSpeed` 赋值语句，而是一行 `setSpeed(0.5 * (1 << level), 20 >> level, 80 >> level)`——这些数在二进制浮点与整数移位下都是精确的，逐位结果与原写法相同。实测把“逐档 switch”换成这一行后，虽然 `adjustSpeed()` 自身从 112 B 涨到 246 B，但整机 Program 从 34970 B 降到 34932 B，净省 38 B。

如果通过 `setSpeed()` 直接写入自定义参数（例如自检脚本或未来的上位机调试），档位名会变成“自定义”（内部记为 -1）；此时 `speedStepDown()` 会从慢速重新开始，保证按键一定能看到变化。

5.3 摇杆的时间门控

摇杆推杆的偏转幅度 `mag`（已扣掉死区）决定“多久走一格”： $\text{interval} = \text{minDelayMs} + \text{mag}/500 \times (\text{fullDelayMs} - \text{minDelayMs})$ 。也就是轻微偏转用最快间隔连续快走，满偏用最慢间隔慢慢走。四路各自独立计时，一路顶到限位或走慢速不会拖累其它路。

这套“偏转越大越慢”的约定同时出现在绘图示教点动（`jogIntervalMs()`）中，两处实现遵循同一原则。之所以反直觉地选择“推得越狠越慢”，是因为手动操作时满偏意味着“要动很多”，这时最容易撞到东西或冲出工作区边缘；把最慢速度留在满偏处，等于把“精确且安全”留给最大幅度的操作。

5.4 绘图的速度规划

绘制时笔尖的线速度上限是 3 单位/秒、加速度上限 6 单位/秒²。每轮按梯形曲线计算本轮速度： $v_{\text{End}} = \sqrt{2 \times \text{加速度} \times \text{剩余距离}}$ 表示“从这一轮开始刹车刚好停在终点”，再与“这一轮能加

到多快”（ $v_{Cur} + \text{加速度} \times dt$ ）取较小值，最后夹到速度上限。得到的曲线是起步加速、中段匀速、终点平稳停住，且不需要预计算速度表。

在此基础上叠加两个约束：① 单次写入位移不超过 0.02 单位（见 2.8.4，代价最小的实现方式只多花 32 B；而“一轮内循环拆小步”要多花 662 B，会使固件超出 Uno 容量）；② 关节角速率 $70^\circ/\text{s}$ 的硬上限，超限时把位移折半重试，最多 4 次。这两条一起保证了“不论轨迹形状如何，每一小步都不会让机械臂猛转”。

过弯策略：折线拐角处不做特意减速，靠 3 单位/秒的温和上限自然过弯；曲线本身连续，不需要额外处理。

5.5 示教点动的速度

示教点动的速度参数

参数	值	说明
步长范围	0.02 ~ 0.25 单位/格	按偏转比例缩放（满偏 = 0.25）
步进间隔	档位的最快 ~ 最慢间隔之间插值	与摇杆同一套约定：偏转越大间隔越长
关节速率上限	$200^\circ/\text{s}$ （DRAW_JOG_MAX_DPS）	比自动绘制宽松：手动操作宁快勿卡
夹爪点动步长	$\text{speed.stepSize} \times 0.5$ 度（默认 0.5° ）	示教期默认关闭（DRAW_JOG_TOOL_ENABLE 0）
生效档位	慢/中/快均可	示教点动也吃 H/L/1/2/3

5.6 取放、回中与回放的速度

其它动作的速度

动作	速度	说明
取放水平/垂直移动	按当前档位：25 / 50 / $90^\circ/\text{s}$	序列中途调档立即生效
取放夹爪开合	$60^\circ/\text{s}$ （固定）	不随档位变化，避免夹取过猛
取放各阶段时长	夹在 180 ~ 4000 ms	防“一步跨太远”与“一步走太久”
绘图回待机（HOME）	$50^\circ/\text{s}$ ，容差 0.4°	关节空间插值，接近目标时判定到位
录制回放	按录制的 200 ms/条推进，播放时拆 4 个 50 ms 子步	时间轴固定，不随档位变化
回中 / 播放前过渡	1500 ms 平滑过渡（BTN_RAMP_MS）	避免从当前姿态突然跳到轨迹起点

5.7 速度相关常量速查

速度相关常量

常量	值	位置
speed 默认（中速）	stepSize 1.0° 、minDelayMs 10、fullDelayMs 40	constant_and_positions.cpp:137-141
档位公式	$0.5 \times (1 \ll \text{level})$ 、 $20 \gg \text{level}$ 、 $80 \gg \text{level}$	constant_and_positions.cpp:839-844
DRAW_V_MAX / DRAW_ACCEL	3.0 单位/秒 / $6.0 \text{ 单位}/\text{秒}^2$	draw_control.cpp 参数区
DRAW_MAX_DPS	$70^\circ/\text{s}$	同上
DRAW_HOME_DPS / DRAW_HOME_TOL_DEG	$50^\circ/\text{s}$ / 0.4°	同上
DRAW_JOG_MAX_DPS	$200^\circ/\text{s}$	同上
DRAW_STEP_MAX	0.02 单位	同上
PICK_RATE_SLOW/NORMAL/FAST_DPS	25 / 50 / 90	pick_place.cpp 参数区
PICK_TOOL_RATE_DPS	$60^\circ/\text{s}$	同上

6 专题：异常处理

本固件的异常处理遵循一条总原则：**宁可不动，不要乱动**。任何“说不清能不能做”的请求都会被拒绝，并且拒绝时保证机械臂的当前状态与之前**完全一致**（Pos 一个字节都不改）。因此系统里没有“半途而废的姿态”，也没有“做了一半的轨迹”。

6.1 分类总览

异常分类与处理方式

类别	典型场景	检测点	处理	对外表现
几何/运动学	点不可达、超出工作空间、反解无解、残差过大	<code>isReachable()</code> 、 <code>getAngleEx()</code> 残差 ≤ 0.05 、分支跳变 $\leq 25^\circ$	拒绝该点/该步；启动前拒绝整条路径	串口 REJECTED；摇杆表现为“到头了”
关节行程	某个关节角超出行程	<code>clampServoAngles()</code> + <code>servoLimit</code>	吸附到最近限位（CLAMP 模式）；整步被视为“到头”	
配置错误	min/max 写反、限位值越界	<code>rangeClampConfig()</code> 、 <code>servoSelfCheck()</code>	就地排序/修正并统计修正条数，不锁死机器	启动/自检输出修正条数
并发冲突	取放中收到绘图命令；绘图时按按键	各模块的 <code>*Busy()</code> / <code>*Locked()</code>	后来者被拒或让位	串口 BUSY；摇杆无反应（让位）
数据异常	录制没位移、缓冲写满、无录制数据	结束录制时的位移判据、 <code>hasRecording()</code>	丢弃本次录制 / 回 EMPTY	串口 DISCARD / EMPTY
协议异常	未知命令、语法错、行太长、给单字符命令加尾巴	<code>protoHandleLine()</code> 解析	回 ERR 且不执行任何动作	串口 ERR
功能门控	命令对应模块未编译，或运行时被关掉	编译开关 + 运行时功能位图	分别回 REJECTED / OFF	串口 REJECTED / OFF
悬挂/卡死	连续多轮没有实际位移；某阶段迟迟走不完	<code>stallGuard()</code> 计数、阶段时长上限	放弃任务回待机 / 夹限时长	任务终止并记录结果码
时序/中断	Timer1 帧长错乱；串口两个读者；一轮循环过长	绝对计数设计、单读者约束、 <code>dtSec</code> 上限 100 ms	设计级规避（见 6.7）	无异常输出（问题在设计上被消除）

6.2 运动学异常

6.2.1 不可达与残差

`isReachable()` 用两个不等式判定：平面半径 $R = \sqrt{x^2 + y^2 + (z - \text{armheight})^2}$ 必须满足 $R \leq L1 + L2 + 1e-9$ 且 $R \geq |L1 - L2| - 1e-9$ 。前者是“够不着”，后者是“太近、折不开”。反解出 4 个候选分支后，每个都用正解回代算残差（位置误差的欧氏距离），取最小者；若最小残差仍大于 0.05，整组丢弃、返回失败。这条残差阈值是“不许糊弄”的体现：反解算出的角度必须真的能把这个点走出来。

6.2.2 分支跳变与吸附

- 分支跳变**：相邻两个采样点单独看都合法，但解出的关节角相差很大（在不同肘上/肘下/反折分支之间切换），机械臂会突然翻一下。检测方式是采样时比较相邻点的 b/r/c，超过 25° （`DRAW_BRANCH_JUMP_DEG` / `PICK_BRANCH_JUMP_DEG`）即判路径不合格。
- 限位吸附**：`getAngleEx()` 会回传 `clamped` 标志，表示“这个点虽然解出来了，但某个关节被硬限位吸附过”。对要求精确到达的调用方（绘图、取放），被吸附一律按失败处理——因为“到达”是假的。

- **启动姿态校验**（取放专有）：启动时手里的关节角与“当前坐标反解出的角”相差不得超过 10° ，否则说明姿态落在另一个分支上，第一段会猛跳，宁可拒绝启动。

6.2.3 角度规范化的边界情形

`normRevAngle()` 负责把角度规整到 $[0^\circ, 360^\circ)$ ，并把“ 360° 减去一个极小量”按 0 处理。没有它，反解在奇异点附近会产出 359.9999° ，被限位误判为越界，从而拒绝一个本来完全合法的姿态。类似地，`ANGLE_EPS = 1e-6` 让 $-1e-15$ 与 -0.0 按 0 处理，`RANGE_EPS = 1e-9` 让“恰好落在边界上”的点被视为可达。这些容差都是为了防止浮点误差变成“机械臂拒绝工作”。

6.3 边界与配置异常

6.3.1 配置写反就地修正

`rangeClampConfig()` 对末端坐标边界的每一轴做排序（若 $\min > \max$ 就交换），返回被修正的轴数；`servoSelfCheck()` 检查四个关节的 $\min \leq \max$ 且都落在 $0 \sim 180$ 内，把不合法项夹回合法范围并返回修正条数。之所以选择“修正而不是报错停机”，是因为这两个值最可能被人工手动改动：修正后机械臂仍可用，且修正条数会出现在输出里，便于发现。

6.3.2 工作空间余量为 0 的含义

`limit` 与“关节行程算出的真实可达包络”完全一致（余量 0 ）。这意味着：软件坐标护栏不会成为限制项——关节行程才是唯一的边界。反过来说，只要一个点的坐标在 `limit` 内，就一定存在一组行程内的关节角能达到它（在分支可达的前提下）。这也解释了为什么 `servoLimit` 的注释里反复强调“这是机械行程的理论上限，不代表不会撞”：四个关节各自到达 180° 的组合里，确实存在下臂向后退回、可能与底座干涉的姿态。因此 `servoLimit` 的注释明确要求“上机请先用慢速档、空载、单步试”。

6.3.3 地面以下

串口坐标命令在解算前把 z 减去肩高（默认 20 ），然后 $z_{\text{内部}} < \text{可达下限}$ 的点直接回 `REJECTED`。也就是说“地面以下”在几何层面就被拒绝了，不会出现“机械臂试图把夹爪插进桌子”的动作。

6.4 忙冲突与让位矩阵

同一时刻只允许一个“源”驱动机械臂。这张矩阵是评审最容易核对的部分——每一格都可以单独测：谁在动时，谁能动（ $\checkmark$ 可用 / $\times$ 被拒 / 让位 = 不响应但不报错）

正在进行的动作 $\rightarrow$	摇杆	串口动作命令	串口 N/R/P/M	取放启动	绘图启动	调速命令
空闲	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$
取放序列中	让位	$\times$ BUSY	$\times$ BUSY	$\times$ BUSY	$\times$ BUSY	$\checkmark$
录制中	$\checkmark$ （正在录的就是它）	$\times$ BUSY	$\checkmark$ （R 收尾 / P 先收尾再播）	$\times$ BUSY	$\times$ BUSY	$\checkmark$
回放 / 回中中	让位	$\times$ BUSY	$\times$ BUSY（播放中按 P 会先停下）	$\times$ BUSY	$\times$ BUSY	$\checkmark$
绘图（含示教）中	示教时由绘图模块自己读（笛卡尔点动）；绘制/暂停时让位	$\times$ BUSY	$\times$ BUSY	$\times$ BUSY	$\times$ BUSY（但 Q/U/W 有效）	$\checkmark$
暂停中	让位	$\times$ BUSY	$\times$ BUSY	$\times$ BUSY	$\checkmark$ （U 继续 / W 取消）	$\checkmark$

唯一的无条件例外是**调速命令**（H/L/1/2/3）：任何状态下都有效。这是安全设计——序列跑得太快时，操作者必须能立刻降速。

6.5 协议与数据异常

协议层异常的判定与回复

情形	判定	回复
未知命令（不在命令表里）	命令分发查找失败	ERR
坐标语法错（如 x,、x10y20）	坐标解析失败	ERR
单字符命令带了尾巴（如 01）	长度检查	ERR
行太长（超过 40 字节）	行缓冲满	丢弃该行（防止缓冲区越界）
模块未编译	编译期开关为 0	REJECTED
模块被运行时关掉（!P/!B/!D）	运行时功能位图	OFF
播放时没有录制数据	buttonHasRecording() 为假	EMPTY
录制结束但不合格	位移 < 10.0 或缓冲写满	DISCARD
动作命令遇到忙碌	各模块 *Busy()	BUSY
坐标点不可达 / 地面以下	反解失败	REJECTED
绘图路径校验失败	validatePath() 返回假	REJECTED

6.6 超时与卡死保护

超时/计数类保护

机制	阈值	作用
串口行静默超时	300 ms	兼容未发送换行的串口助手：静默即视为整行到齐
单次推进可用时长	最多 50 ms (DRAW_TICK_MAX_MS)	防止串口打印卡顿后，一轮突然跨出很大一步
路径跟随的位移时长	夹在 5 ~ 100 ms (applyDtSec())	示教点动是事件驱动的（10~40 ms 一次），按循环周期算会被整条拒绝；上限防长时间暂停后的第一轮大跳
折半重试次数	最多 4 次	关节速率超限时逐步缩小步长；仍失败则本轮不动
绘图悬挂计数	连续 500 轮无位移 (DRAW_STALL_LIMIT)	放弃任务并回待机，避免“既不完成也不报错”
取放单阶段时长	夹在 180 ~ 4000 ms	防止某段无限制地慢慢走
示教自动开始延时	700 ms	5 点记满后留一点检查时间
播放/回中过渡	1500 ms	从当前姿态平滑进入轨迹起点

6.7 时序与中断相关的“设计级规避”

有一类异常不是“检测到再处理”，而是在设计上让它不可能发生。三处值得单独说明：

- 1) **Timer1 帧长错乱**：帧尾比较值在“4 路都发完”的那次中断里算好绝对计数；若等到帧尾中断里现算，TCNT1 已越过 40000，uint16 下溢会把下一帧推到 52.8 ms 之后（19 Hz 且抖动）。见 2.2.3。
- 2) **串口两个读者**：一行命令被两个函数各读一半，行缓冲永远拼不出完整指令——历史上真实发生过（handleSerialSpeedCmd() 与协议模块并存）。现在头文件里明确写了“本模块是唯一读者”，旧函数已删除。
- 3) **长循环导致的步长失控**：dtSec 取“距上一次真正写入点位的真实时长”并夹在 5~100 ms。若直接用循环周期，遇到串口打印卡顿、或示教点动这种事件驱动路径，就会算出过大的步长或过小的步长。

另外，`millis()` 是 32 位无符号数，比较时间差时一律用无符号减法（`now - last`），因此计数器回绕不会产生“负的时间”。所有时间门控都遵循这个写法。

6.8 为什么不用异常机制

AVR 平台没有 C++ 异常支持，强行引入会带来可观的代码体积与栈开销（当前只剩 600 B flash 余量）。更重要的是，本固件的异常都是**可预期的几何/状态问题**，而不是程序错误：它们的正确处理方式是“拒绝并保持状态不变”，这用返回值表达更直接，也更容易被自检探针逐条覆盖。

因此所有对外接口都遵循同一约定：**返回值表达结果，失败时状态不变**。这条约定使得“错误路径”可以被穷举测试（探针直接构造不可达点、超行程角、无录制数据的播放请求等），而不是靠人工上机碰运气。

7 未完成功能与原因分析

本章如实列出当前版本“没有做到”的部分，并分析原因是容量约束、硬件条件还是设计取舍，以及若要补上需要什么。这些内容同样出现在发布说明的“已知限制”一节。

7.1 真机标定值与上机验证缺失

未实测的参数

参数	当前值	为什么是默认值	补上需要什么
取放台面高度 PICK_GRASP_Z	12.0	理论值由几何算得，未与真实台面高度对齐	量出夹爪闭合时中心离地高度，改一个常量
绘图纸面高度 DRAW_PEN_Z	12.0	同上；已提供 p 命令现场设定，无需重编译	放一张纸，用 p 命令试到笔尖刚好落纸
图形半宽 / 中心 (DRAW_HALF/DRAW_CENTER_*)	6.0 / (20,0)	按默认纸张尺寸估计	n / o 命令现场设定
摇杆方向与中位	按实机接线，中位自标定	方向已按上机观感调过一次（上臂/下臂），其余未复核	接通后逐路推杆确认，必要时改反向开关

根本原因：全部结论都来自主机侧的编译、静态分析与 PC 探针（模拟 Arduino API 的自检环境），**没有在真实 Uno + 舵机 + 摇杆上跑过**。这也是为什么 servoLimit 的注释里反复强调“上机请先慢速、空载、单步试”。**影响：**几何算法与状态机逻辑有自检覆盖，但“机械装配方向、舵机行程实测值、摇杆手感”三类问题只能在实机上暴露。**若要补齐：**接上硬件后按附录 B 的检查清单逐项核对，本固件已把最可能需要改的量都做成了常量或运行期命令。

7.2 物理按键默认关闭

WEARM_BUTTON_PINS 默认为 0，即完全不读 D2~D5（连 pinMode 都不做）。这不是“没实现”，而是实测后的主动关闭：曾经把 KY-023 摇杆模块的 SW 脚接到 D2，结果“把右摇杆推到最前”时机械结构会压到那枚轻触开关，产生一次假的按键事件——正在手动推杆的操作会被“循环取放”抢走（摇杆随即被锁定），非常危险。

影响：四个动作（循环取放/录制/播放/回中）仍然全部可用，只是必须从串口发 N/R/P/M/0。**若要恢复：**把按键接到不会与机械行程干涉的位置，并把开关改为 1；注意 D13 已被 LED 占用、D6~D9 已被舵机占用，可用引脚有限，必要时需要重新分配。

7.3 flash 只剩 600 B

默认配置（三功能全开）占 31656 B，可用 32256 B，**余量 600 B**。这是当前版本最硬的约束，直接决定了 7.4 与若干细节实现方式（例如 2.8.4 里“一轮只走一步”而非循环拆步，正是因为它省 630 B）。

若要腾空间，按“收益/风险”排序的候选：① 关掉绘图内置的曲线（Catmull-Rom 与弧长测量约占 2 KB）；② 关掉某个功能模块（编译开关，收益 3.2~12.5 KB，代价是功能缺失）；③ 继续逐函数压缩（历史上 v1.1.0 用这种方式省下约 1 KB，但边际收益递减）；④ 换用 flash 更大的板子（Mega/ESP32），这是唯一无功能损失的方案。

7.4 绘图任务从 7 个减到 4 个

内置图形原有 7 个（直线、字母 V、字母 N、三角形、字母 Z，加两种示教任务），现只保留直线与字母 V。删掉三个图形的原因：它们都要各自占用一张 PROGMEM 顶点表，而当前版本同时把整体速度放慢（3 单位/秒）、采样步长减半（0.5 → 0.25）、新增单步位移上限——四项合计只在默认配置下净省 64 B。也就是说“删图形”省下的空间刚好被“画得更稳”吃掉，功能换的是质量。

若要加回：每个简单折线图形约 100~200 B 顶点表 + 少量代码；在当前余量下需要先省出空间。

7.5 没有速度环与位置反馈

四个舵机是开环控制的：固件按“时间 × 设定速度”估算当前姿态，没有任何编码器或电位器反馈（Pos.ser 是命令值，不是测量值）。**根本原因：**MeArm 套件用的是普通舵机（无反馈脚）。**影响：**① 负载较大时实际速度低于设定值，笔尖轨迹会有跟随误差；② 无法检测堵转；③ “回中”依赖舵机重复定位精度。**若要补齐：**换用带反馈的舵机并把反馈接进 ADC，或者引入电流检测判断堵转；这属于硬件层面的改造。

7.6 其它未实现的扩展

未实现的扩展与原因

扩展	现状	原因/若要做的路径
串口位置遥测（读回当前 x/y/z）	没有遥测命令；只有！查询功能开关	省 flash 与协议面；加一条 T 命令约几十字节，容易补
多物体取放队列	一次只启动一个 A/B/C	状态机本就非阻塞，扩展为队列会引入“排队失败”的处理路径；当前用连续发命令即可近似
断电续绘 / 断点续放	不支持	需要非易失存储与“上次执行到哪”的持久化；Uno 的 EEPROM 只有 1 KB 且擦写寿命有限
SD 卡 / 上位机 GUI	无；绘图任务只能在固件内选	超出 Uno 的引脚与内存预算；更合理的做法是上位机把轨迹算好，用串口坐标命令下发（现有协议已支持）
坐标系旋转 / 末端姿态控制	坐标系固定为基座朝 +x 的右手系	4 自由度里 angle4 不参与反解，独立姿态控制需要额外自由度
曲线类型可选（B 样条、贝塞尔）	只有向心 Catmull-Rom	每种都占 flash；现有类型已满足“经过示教点且圆滑”的需求
单文件版与模块化版体积完全一致	单文件版多 20 B	LTO 对“同一编译单元”与“多个编译单元”的内联决策不同（main +24 B、getAngleEx +8 B、clampServoAngles -20 B）；功能一致，20 B 无实际影响

8 评审速查

8.1 一分钟了解全貌

- 状态只有一个：Pos.ser（四个关节角）；末端坐标是算出来的。
- 两个入口：moveJointStep()（走角度，摇杆用）与 moveToPoint()（走坐标，绘图/取放/串口用）。
- 三层速度：档位（0.5/1/2 度每格）→ 各动作规划（摇杆插值、绘图梯形）→ 关节速率硬上限（70°/s）。
- 四道安全网：两级边界（关节行程 + 坐标范围）→ 启动前整条路径采样校验 → 每步整点拒绝/整步回退 → 连续 500 轮无位移就放弃任务。
- 五条外部通道：两根摇杆（4 路）、串口命令、运行时功能开关（!P/!B/!D）、绘图标定（p/n/o）、录制回放（R/P）。

8.2 关键常量表

评审最可能抽查的常量

常量	值	位置
L1 / L2 / armheight	20.0 / 20.0 / 0.0	constant_and_positions. cpp:37
POS_HOME	(20, 0, 20)	constant_and_positions. cpp:29
limit	$x, y \in [-40, 40]$; $z \in [-20, 40]$	constant_and_positions. cpp:130-134
servoLimit	$b/r/c \in [0, 180]$; $f \in [60, 150]$	constant_and_positions. cpp:147-159
servoLimitMode	SERVO_LIMIT_CLAMP（吸附不报错）	constant_and_positions. cpp:162
速度默认 / 档位	中速 1.0°/10ms/40ms；慢 0.5°/20/80； 快 2.0°/5/20	constant_and_positions. cpp:137-141、:839-844
WEARM_SHOULDER_HEIGHT	20.0（串口地面坐标系与内部坐标系的唯一 差值）	serial_protocol.h:50-52
PROTO_BAUD / 行缓冲 / 超时	115200 / 40 字节 / 300 ms	protocol_constants.h:85-89
录制	192 条 × 4 B = 768 B；每 200 ms 一条； 上限 38.4 s；单位 0.5°	weArm_config.h、 button_control.cpp:87-93
取放	台面 12.0、飞行高度 6.0、停顿 400 ms、 档位速度 25/50/90°/s	pick_place.cpp 参数区
绘图	笔速 3.0、加速度 6.0、关节上限 70°/s、 单步 0.02、纸面 12.0、半宽 6.0	draw_control.cpp:66-124
DRAW_TEACH_MAX_POINTS	5	draw_control.h

8.3 源码索引（抽查用）

想核对什么 → 看哪个文件的哪一部分

想核对	文件与位置
姿态状态、边界、正/逆解	constant_and_positions.h:1-244; constant_and_positions.cpp:349-543
摇杆步进与坐标运动两条入口	move.cpp（moveJointStep 与 moveToPoint）

路径校验的四个核心函数	path_core.h:1-105
舵机时序与脉宽映射	servo_drive.h:1-75（说明）+ servo_drive.cpp（纯状态机）
取放阶段与位置表	pick_place.cpp:1-150（阶段名、参数、位置表）
摇杆映射、标定、让位	joystick_control.h:1-133; joystick_control.cpp:181-435
录制/回放/回中	button_control.cpp:215-611
绘图状态机、曲线、示教点动、速度规划	draw_control.cpp:66-124（参数）、:308-353（曲线）、:431-499（校验）、:509-560（速度）、:715-883（推进与点动）
串口命令表与回复语义	serial_protocol.h:1-128; protocol_constants.h:1-103
编译开关与容量取舍	weArm_config.h:1-226
主循环与舵机镜像	meArm.ino（setup/loop/writeServo）
单文件生成器	.selfcheck/make_single.ps1; 产物 single/meArm/meArm.ino

8.4 验证方法与可复现证据

本固件配套一整套 PC 端自检：它用 mock 的 Arduino API 把固件源码直接编译到主机上运行，因此几何算法与状态机可以在没有硬件的情况下被逐条断言。

自检与验证入口

脚本	做什么	当前结果
.selfcheck/run_all.ps1	编译 9 个固件编译单元 + 运行 11 个探针	9 个 TU 零警告；11 个探针 ALL PASS
.selfcheck/avr_build.ps1	用真实 avr-gcc 编译并报告 flash/SRAM 占用	默认配置 31656 B (96.6%) / 1415 B (69.1%)
.selfcheck/avr_build.ps1 - ExtraDefs @(...)	编译其它功能组合	四种组合数据见 1.4 节
.selfcheck/out/kin_verify.py	20 万组随机关节角做“正解→反解→正解”往返	0 组失败，最大位置误差 2.266e-10
.selfcheck/make_single.ps1	从模块化源码生成单文件 .ino（幂等）	332015 B，与入库产物逐字节一致
avr_build.ps1 -SketchDir single/meArm	编译单文件版	31676 B / 1415 B（模块化版 31656 B）

11 个探针的分工：probe_axes（轴映射与行程）、probe_button（录制/回放/回中，含 38.4 s 上限的边界用例）、probe_draw（绘图状态机、曲线、示教、校验）、probe_joystick（摇杆映射、死区、标定）、probe_move（两种运动入口与三种模式）、probe_pick_place（取放阶段、位置表约束）、probe_pins_off（按键关闭配置下的行为）、probe_protocol（13 段串口协议回归，含运行时开关与忙守卫）、probe_rt（运行时功能位图）、probe_servo_drive（脉冲时序纯状态机）、wearm_ino_test（主 sketch 的初始化与调度顺序）。

几个具体证据点：① 录制上限：probe_button 的边界用例显示 38.2 s 时命中 190/192 条记录，38.8 s 时判定超限并被丢弃；② 取放位置表：探针断言“放置点离其它物体初始点”的最小距离为 7.2111（见 2.5.2）；③ 单文件版：生成器重跑两次的 SHA256 完全一致，且与仓库中入库的文件一致（幂等性可见证）。

附录 A 术语表

术语

术语	含义
关节角 / 角度模式	直接控制四个舵机角度 (b/r/c/f)，是固件的主控制方式
末端坐标 / 坐标模式	控制夹爪中心在空间中的位置 (x, y, z)，由反解换算成角度
内部坐标系	原点在肩关节；Pos.rec 用的就是它
用户坐标系 / 地面坐标系	原点在“过肩关节的地面垂足”；串口命令用它，比内部坐标高一个肩高（默认 20）
两级边界	servoLimit（关节行程，最终防线）与 limit（末端坐标范围，粗护栏）
反折分支	两连杆问题的第二种解（平面坐标取负），没有它则基座转过 90° 后大片区域无解
残差	反解出的角度用正解回代后与目标点的距离；超过 0.05 即判失败
分支跳变	相邻采样点解出不同分支，关节角突变；超过 25° 即判路径不合格
吸附 (clamped)	反解结果被关节限位夹住，意味着这一点其实到不了
梯形速度	加速—匀速—减速的速度曲线；由“刹车距离”公式自然得到，无需预计算速度表
向心 Catmull-Rom	以点间距离的平方根为节点间距的样条；经过控制点且不生尖角/打圈
弧长参数化	按“走过的实际距离”而不是参数 u 推进曲线，使笔速均匀
示教点动	示教时用摇杆直接推动笔尖坐标（而不是关节角）
让位	某个模块正在工作时，另一个模块不响应（但不报错）
脱离（挂起）/ 悬挂	任务既没完成也没报错地空转；由 stallGuard() 检测并终止
LTO	链接期优化；本工程开启，因此“局部更小”的改动可能让全程序变大（见 2.3.3）

附录 B 复现步骤（评审可直接执行）

以下命令在仓库根目录执行（Windows PowerShell）。① 全量自检：

```
powershell -NoProfile -ExecutionPolicy Bypass -File .selfcheck\run_all.ps1 → 9 个编译单元零警告 + 11 个探针全部 PASS
```

② 真实 AVR 编译与容量：

```
powershell -NoProfile -ExecutionPolicy Bypass -File .selfcheck\avr_build.ps1 → Program 31656 B / Data 1415 B
```

③ 其它功能组合（示例：关掉按键模块）：

```
powershell -NoProfile -ExecutionPolicy Bypass -File .selfcheck\avr_build.ps1 -ExtraDefs @('-DWEARM_ENABLE_BUTTONS=0')
```

④ 运动学往返验证（需要 Python + 无第三方依赖）：

```
python .selfcheck\out\kin_verify.py → 20 万组 0 失败（报告写入同目录文本文件）
```

⑤ 单文件版生成与编译：

```
powershell -NoProfile -ExecutionPolicy Bypass -File .selfcheck\make_single.ps1, 随后 avr_build.ps1 -SketchDir single\meArm
```

上机前建议的检查清单（按顺序，全部在慢速档、空载、机械臂抬离台面的条件下进行）：

- 1) 摇杆中位：上电后不动摇杆，确认机械臂不漂移；若漂移，检查接线与 JOY_DEADZONE。
- 2) 方向：逐路轻推，确认基座/上臂/下臂/夹爪的方向与 2.6.1 的表一致；不一致则改对应的反向开关。

- 3) 行程：用摇杆把每个关节慢慢推到头，确认在机械限位之前就停住（`servoLimit` 已按理论值设定，可能需要按实机微调）。
- 4) 夹爪：确认 `O`（张开）与 `S`（闭合）到达的角度不会顶死舵机。
- 5) 纸面标定：用 `p` 命令逐步找到“笔尖刚好落纸”的高度；`n/o` 设定图形大小与位置。
- 6) 取放：把物体放在表中位置，发 `A`，全程盯住，必要时立即 `L` 降档。
- 7) 录制回放：发 `R` 手动做一段动作，再发 `R` 结束，发 `P` 回放，确认终点姿态对得上。

最后一句：本文档描述的是 `v1.6.3`（tag `7a1d818`）的行为；单文件整合产物、运动学推导文档（`docs/kinematics_derivation.docx`）与本说明同属一个交付批次。